

DEPL Push Gateway 개발 가이드

작성일: 2026-09-09 | 용도: 기능 검토·수정 설계·개발 인수인계

1. 문서 기준과 시스템 범위

현재 로컬 소스를 직접 확인한 개발 가이드입니다. 원격 기준 커밋은 42133d6이고, 로컬에 적용된 com.depl.push 패키지 변경을 포함합니다. 이번 문서 작성에서는 기능 수정이나 운영 환경 변경을 수행하지 않았습니다.

항목	현재 구현
서버	Spring Boot 3.5.16 / Java 21
발송 라이브러리	Firebase Admin Java SDK 9.10.0
저장소	MariaDB / Spring JDBC / Flyway V1~V2
Java 패키지·Maven groupId	com.depl.push
시작 클래스	com.depl.push.PushGatewayApplication
전달 방식	DB Outbox polling / FCM Topic
앱 연결	인터넷망 앱에서 직접 Firebase 등록·구독

Gateway는 업무 이벤트를 큐에 담고, 사용자·부서·공지 Topic으로 고정 8종 알림을 발송합니다. QR 생성과 관리자 테스트 화면도 제공합니다. RabbitMQ나 Redis는 현재 사용하지 않습니다.

스마트폰이 Gateway에 직접 접속하지 않는 구조입니다. 업무 상세 조회, 앱 수신 확인 응답, 읽음 상태 동기화, 누락 내역 복원용 앱 API는 현재 제공하지 않습니다. 서버 history는 단말 수신 이력이 아니라 FCM 접수 이력입니다.

읽는 순서

- 2장: 기능별 코드 위치와 수정 영향
- 3장: 큐 선점·발송·재시도·중복 처리
- 4장: 테이블과 마이그레이션
- 5장: 업무 연동·알림 Payload 계약
- 6장: QR·관리자 화면·인증
- 7장: 실행 설정과 검증 방법
- 8~9장: 코드에서 확인한 검토 항목과 변경 제안

검토 항목은 현재 코드의 관찰 결과와 수정 제안을 구분했습니다. 운영 장애 재현이나 부하 테스트를 완료했다는 의미는 아닙니다. 실제 키·토큰·DB 접속 비밀번호는 문서에 포함하지 않았습니다.

2. 기능별 코드 지도

아래 Java 파일은 push-gateway/src/main/java/com/depl/push/ 를 기준으로 한 상대 경로입니다. 테스트는 src/test/java/com/depl/push/ 에 있습니다.

수정하려는 기능	주요 파일·위치
실행·설정 바인딩	PushGatewayApplication.java / config/PushProperties.java
주기·worker ID	service/PushDispatchScheduler.java
업무 큐 등록	service/PushQueueService.java
대상별 허용 타입	service/PushTargetPolicy.java
선점·RETRY·DEAD	repository/PushQueueRepository.java
처리 순서·오류 분기	service/PushProcessor.java
재시도 횟수·간격	service/RetryPolicy.java
성공 이력·큐 삭제	repository/PushCompletionRepository.java
제목·본문·문구 버전	template/NotificationTemplateFactory.java
업무 타입·앱 타입 매핑	domain/NotificationType.java
FCM·APNs Payload	sender/FirebasePushSender.java
Topic 생성·조회	service/PushTopicService.java / repository/PushTopicBindingRepository.java
QR 서명·PNG 생성	service/EnrollmentQrService.java
업무 사용자·부서 검색	service/BusinessDirectory.java / config/BusinessDirectoryConfiguration.java
관리자 화면·입력	api/PushTestController.java / resources/templates/
인증·요청 제한·지표	config/SecurityConfiguration.java / service/PushTestRateLimiter.java / config/MetricsConfiguration.java

변경 범위 판단

기존 타입의 문구만 바꾸면 템플릿과 관련 테스트를 우선 수정합니다. 새 타입 추가는 enum, 대상 정책, DB check 제약, 앱의 허용 타입 목록까지 영향이 있습니다. DB 변경은 src/main/resources/db/migration/에 새 버전 파일을 추가합니다.

Maven groupId는 pom.xml, 환경변수 기본값은 application.yml에서 관리합니다. Gateway 패키지는 Android applicationId나 iOS Bundle ID와 별개입니다. Java 패키지 변경만으로 모바일 앱을 다시 등록하지는 않습니다.

3. 큐 처리와 전달 보장

1. 업무 처리와 같은 DB 트랜잭션에서 PushQueueService가 PENDING 행을 삽입합니다.
2. Scheduler가 processBatch(workerId)를 호출합니다. fixedDelay이므로 한 배치가 끝난 뒤 기본 30초를 기다립니다. 정확히 매 시각 30초마다 시작하는 방식은 아닙니다.
3. claimBatch()의 짧은 트랜잭션에서 오래된 PROCESSING을 RETRY로 돌리고, 처리 시각이 된 행을 FOR UPDATE SKIP LOCKED로 가져옵니다. 기본 100건을 한 번에 PROCESSING으로 표시합니다.
4. 선점 트랜잭션 종료 후 각 행을 순차 처리합니다. Topic을 조회하고 템플릿을 만든 뒤 FCM을 호출합니다. 네트워크 요청 동안 DB 행 잠금을 계속 유지하지 않습니다.
5. 접수 성공 시 completeAccepted()가 history INSERT와 queue DELETE를 하나의 트랜잭션으로 수행합니다. 소유 worker가 일치하는 큐 1건을 삭제하지 못하면 예외로 롤백합니다.
6. 실패하면 오류 유형과 실패 횟수에 따라 RETRY 또는 DEAD로 바꿉니다. 미처리 예외도 재시도 처리 대상으로 들어갑니다.

결과	현재 처리
ACCEPTED	FCM_ACCEPTED 이력 저장 후 큐 삭제
UNAVAILABLE / INTERNAL / QUOTA_EXCEEDED	재시도 가능 오류
INVALID_ARGUMENT / UNREGISTERED / SENDER_ID_MISMATCH	영구 실패로 DEAD
THIRD_PARTY_AUTH_ERROR	인증 실패로 DEAD
Topic 누락·비활성	TOPIC_BINDING_MISSING_OR_DISABLED로 DEAD
알려지지 않은 발송 오류	재시도 가능 오류

재시도는 기본 최대 5회 실패까지이며 지연은 30초, 2분, 5분, 15분입니다. 실제 재처리 시점에는 polling 대기과 앞선 배치 처리 시간이 추가됩니다. 모든 인증 문제를 동일하게 식별하는 것은 아니며 SDK가 분류하지 못한 오류는 재시도될 수 있습니다.

중복과 누락의 경계

FCM 접수 직후 프로세스 종료나 DB 실패가 생기면 같은 eventId가 다시 발송될 수 있습니다. DB 완료 처리는 원자적이지만 FCM과 DB를 묶는 분산 트랜잭션은 없습니다. 재시도 기반 전달이며 정확히 한 번 표시를 보장하지 않습니다.

앱 중복 제거도 로컬 내역 보관 범위 내에서 동작합니다. 특히 iOS는 시스템 APNs 알림이 앱 처리 전에 표시될 수 있어 앱 내 중복 제거만으로 모든 시스템 알림 중복을 막는다고 볼 수 없습니다. DEAD 처리된 항목의 최종 전달 또한 보장되지 않습니다.

근거: PushDispatchScheduler.dispatch, PushQueueRepository.claimBatch, PushProcessor.processOne, PushCompletionRepository.completeAccepted, RetryPolicy.afterFailure.

4. DB 설계와 이력의 의미

테이블	주요 컬럼·역할
push_queue	event_id, target_type/id, notification_type, actor_name, status, attempt_count, next_attempt_at, locked_by/at, last_error_code
push_send_history	event_id, target_type/id, notification_type, actor_name, platform, firebase_message_id, result_code, requested_at, sent_at
push_topic_binding	target_type/id와 topic_name 연결, enabled 상태
push_device	레거시 토큰 등록 테이블. 신규 Topic 앱에서는 미사용

queue와 history에는 각각 event_id unique 제약이 있습니다. 두 테이블 사이를 아우르는 단일 unique 제약은 아닙니다. 완료 후 같은 eventId를 다시 큐에 넣는 상황까지 먹등하게 처리하는 전역 이벤트 원장은 현재 없습니다.

Topic에는 (target_type, target_id) 및 topic_name unique 제약이 있습니다. USER/DEPARTMENT Topic은 난수이며 NOTICE는 ALL / notice_all로 고정됩니다. 테이블의 enabled를 끄는 것은 Gateway 발송 조회를 제한하는 것이며 단말 구독 자체를 강제로 해지하지 않습니다.

V1과 V2

V1은 큐·완료 이력·레거시 기기를 만들고 타입·상태 제약을 정의합니다. V2는 Topic binding을 추가하고 queue/history에 target_type, target_id, delivery_channel을 추가합니다. 기존 recipient_user_id 값은 USER 대상으로 backfill됩니다.

이미 적용한 V1/V2 파일은 수정하지 않고 V3 이상의 새 마이그레이션을 작성해야 합니다. 새 notification_type을 추가하면 V1에서 만든 ck_push_queue_type 제약도 새 마이그레이션으로 확장해야 합니다.

현재 이력에 저장되지 않는 값

전송된 title/body/templateVersion, 성공까지의 전체 시도 횟수, 원래 큐 생성 시각, Firebase 프로젝트 ID, 개별 단말 수신·읽음 여부는 완료 이력에 없습니다. 제목과 본문은 매 시도 때 템플릿으로 다시 생성됩니다.

따라서 템플릿 배포 후 재시도 문구가 달라질 수 있고, 과거 전송 문구를 이력만으로 정확히 복원할 수 없습니다. 감사 목적이 필요하면 내용 스냅샷, 버전, 보관 기간과 민감정보 정책을 먼저 결정하세요.

마이그레이션 검증 기준

빈 DB 설치, V1 데이터의 V2 전환, 기존 이력 조회, 새 타입 입력, 롤백 호환성을 확인합니다. 회사 Firebase로 전환할 때 같은 큐를 사용하면 남은 이벤트가 새 프로젝트로 발송될 수 있으므로 발송 중지·대기 큐 처리 계획이 필요합니다.

근거: V1__create_push_tables.sql, V2__topic_delivery.sql, PushQueueRepository.insert, PushCompletionRepository.completeAccepted.

5. 업무 연동과 메시지 계약

업무 시스템과 Gateway가 별도 JVM이라면 Gateway의 Java 서비스를 직접 호출할 수 없습니다. 현재 문서의 서비스 호출 예시는 같은 Spring 애플리케이션·DataSource·트랜잭션 매니저에서 사용한다는 전제입니다. 별도 업무 서버에는 큐 삽입 모듈 또는 검증된 SQL 연동이 필요합니다. 이때도 업무 변경과 큐 삽입이 같은 DB 트랜잭션이어야 합니다.

```
@Transactional
public void onTaskAssigned(String assigneeld) {
    // 업무 데이터 변경을 같은 트랜잭션에서 수행
    pushQueueService.enqueueUser(
        assigneeld, NotificationType.TASK_ARRIVED, null);
}
```

대상	운영 허용 타입	관리자 테스트
USER	모든 8개 타입	모든 타입
DEPARTMENT	SOURCE_OVERLAP, NOTICE_CREATED, TASK_ARRIVED	모든 타입
NOTICE	NOTICE_CREATED만	모든 타입

테스트 화면은 enqueueForTest()로 대상 정책만 완화합니다. 운영 경로와 달리 부서·공지에도 8개 타입을 테스트할 수 있습니다. 사용자·부서 ID는 trim 후 최대 128자이며, NOTICE ID는 ALL로 정규화됩니다.

TASK_MENTIONED와 COMMENT_MENTIONED는 actorName이 필수입니다. 서비스에서는 NFKC 정규화, 제어 문자·연속 공백 정리 후 50자로 자릅니다. 관리자 폼은 50자 초과 입력을 검증 오류로 처리합니다. 나머지 타입은 actorName을 허용하지 않습니다.

앱으로 보내는 data

eventId(UUID 문자열), notificationType, title, body, templateVersion을 보냅니다. Android는 data-only와 HIGH 우선순위, iOS는 같은 data에 APNs alert·기본 소리·priority 10을 추가합니다. actorName은 본문 안에만 반영합니다.

업무 타입과 앱 타입의 차이: TASK_COMMENT_CREATED → COMMENT_ADDED, SOURCE_OVERLAP → SOURCE_CONFLICT, NOTICE_CREATED → NOTICE_REGISTERD. 나머지 타입 문자열은 같습니다.

기존 타입 문구 변경은 템플릿 중심으로 가능하지만 새 타입은 앱의 허용 목록도 갱신해야 합니다. 구버전 앱이 알 수 없는 타입을 버리는 점을 고려해 앱 배포 후 새 타입 발송 순서를 정하세요.

근거: PushQueueService, PushTargetPolicy, NotificationType.wireName, NotificationTemplateFactory, FirebasePushSender.buildMessage.

6. QR·관리자 화면·인증

QR은 version, firebaseProjectId, topics(user/department/notice), issuedAt, expiresAt, nonce, signature를 포함합니다. 기본 TTL은 3분, 최대 10분입니다. 공개키를 가진 앱이 검증한 뒤 Firebase에 직접 구독합니다.

서명 입력 순서는 다음과 같습니다. JSON 필드 순서 대신 아래 canonical 문자열을 UTF-8로 사용합니다.

```
version|firebaseProjectId|userTopic|departmentTopic|
noticeTopic|issuedAt|expiresAt|nonce
```

위 두 줄은 지면상 줄바꿈입니다. 실제 입력은 개행 없는 한 줄입니다. ECDSA P-256 / SHA256withECDSA, DER signature Base64, X9.63 공개키 Base64를 사용합니다. 서버 개인키는 PKCS#8 PEM이며 공개키 출력 메서드는 같은 경로의 .pub 파일을 읽습니다.

Method / 경로	역할
GET /internal/push-test/enrollment	사용자·부서 QR 등록 화면
GET /internal/push-test/qr	userId/departmentId로 QR PNG 생성, no-store
GET /internal/push-test/messages	대상 검색·타입 선택 테스트 폼
POST /internal/push-test/send	form 입력 검증, 큐 등록, 결과로 redirect
GET /internal/push-test/events/{eventid}	큐 또는 완료 이력 표시
GET /api/push/history?limit=50	Principal 기반 최근 이력, 1~100으로 보정
POST / DELETE /api/push/devices(/..)	레거시 기기 등록·해지, 신규 앱 미사용

권한과 회사 연동 경계

테스트 페이지는 기본 비활성입니다. 활성화 시 /internal/**는 PUSH_ADMIN 또는 PUSH_TESTER 권한을 요구합니다. 무인증 옵션은 remoteAddr의 loopback 여부를 검사합니다. 로컬 프록시를 통해 외부에 노출하면 이 전제를 깨뜨릴 수 있습니다.

부서·공지 발송은 확인 체크가 필수이고 기본 분당 10회 제한은 인스턴스 메모리 기준입니다. 다른 인스턴스로 우회하거나 재시작하면 전역 제한이 유지되지 않습니다.

BusinessDirectory 기본 구현은 빈 목록입니다. 회사 사용자·부서 조회 Bean과 업무 인증 연동이 필요합니다. QR은 관리자가 입력한 ID로 발급되며 개인 사용자가 스스로 등록하는 화면에서는 인증 사용자와 실제 부서를 서버에서 결정하도록 별도 구현해야 합니다.

근거: EnrollmentQrService, PushTestController, SecurityConfiguration, BusinessDirectoryConfiguration, PushTestRateLimiter. 상세 응답 명세: docs/API.md.

7. 실행 설정과 개발 검증

환경변수	기본값 / 역할
DB_URL / DB_USERNAME / DB_PASSWORD	MariaDB 접속, 실제 값은 외부 주입
PUSH_SCHEDULER_ENABLED	true
FIREBASE_ENABLED	false. true이고 scheduler도 켜져야 자동 발송
PUSH_SCHEDULER_DELAY / PUSH_BATCH_SIZE	30s / 100건
PUSH_PROCESSING_TIMEOUT / PUSH_MAX_ATTEMPTS	5m / 5회
FIREBASE_PROJECT_ID	빈 값. 회사/앱/QR 프로젝트와 일치 필요
GOOGLE_APPLICATION_CREDENTIALS	서비스 계정 JSON 파일 경로
PUSH_QR_PRIVATE_KEY_PATH / PUSH_QR_TTL	개인키 경로 / 3m
PUSH_TEST_PAGE_ENABLED	false
PUSH_TEST_PAGE_AUTHENTICATION_ENABLED	true
PUSH_TEST_PAGE_USERNAME / PUSH_TEST_PAGE_PASSWORD_HASH	계정 / BCrypt 해시
PUSH_TEST_ENQUEUE_WHEN_FIREBASE_DISABLED	true. 비활성 시 큐에 적재만 가능
PUSH_TEST_REQUESTS_PER_MINUTE	10

Firebase 비활성 상태에서는 자동 Scheduler Bean이 만들어지지 않습니다. 테스트로 쌓인 큐는 나중에 Firebase를 활성화하면 발송 대상이 될 수 있습니다. DisabledPushSender 자체를 직접 호출하면 FCM_DISABLED 실패를 반환합니다.

검증 명령과 범위

```
cd push-gateway
# JAVA_HOME은 설치된 JDK 21 경로로 지정
mvn clean verify
java -jar target/push-gateway-0.1.0-SNAPSHOT.jar
```

이전 2026-09-07 패키지 변경 검증에서 Java 21, 43개 테스트, 실행 JAR 빌드가 통과했습니다. 이번 문서 작업에서는 테스트를 재실행하지 않았습니다. MariaDB 통합 테스트 10개는 Docker가 없으면 skip될 수 있으므로 성공 문구뿐 아니라 skipped 수를 확인하세요.

관측 지표: push.queue.waiting, push.queue.dead, push.queue.waiting.by_target, push.process.success/retry/dead, push.fcm.requests, push.fcm.request, push.topic.resolution.failure. health/info/metrics/prometheus가 노출 대상으로 설정됩니다.

서비스 계정 JSON 인증 구성의 외부 연결은 fcm.googleapis.com과 oauth2.googleapis.com의 TCP 443입니다. 사내 DNS·NTP·DB는 별도입니다. 조건부 주소·회사 전환: docs/COMPANY-DEPLOYMENT.md.

8. 우선 검토할 정확성·운영 항목

다음은 코드에서 확인한 동작입니다. 우선순위는 운영 전 검토 제안이며, 이 문서에서 수정하거나 장애를 실기기로 재현하지 않았습니다.

우선 1: 사용자 이력 대상 분리

관찰: PushCompletionRepository.findRecentForUser()는 recipient_user_id만 비교합니다. V2 입력은 USER뿐 아니라 DEPARTMENT와 NOTICE의 targetId도 이 컬럼에 저장합니다.

영향: 사용자 ID와 부서 ID가 같거나 사용자 ID가 ALL이면 다른 대상 유형 이력이 섞일 수 있습니다. API 문서의 “USER 전송 이력” 설명과 SQL이 일치하지 않습니다.

제안: target_type='USER' AND target_id=?로 조회하고, 사용자·부서 ID가 같은 테스트를 추가합니다. 부서 이력을 포함하려면 실제 소속 권한 조회를 별도로 설계합니다.

우선 1: 선점 만료와 재전송 상한

관찰: claimBatch()는 최대 100건을 한 번에 같은 시각으로 잡고 Processor가 순차 발송합니다. 만료 시 heartbeat 없이 복구하며 stale 복구는 attempt_count를 증가시키지 않습니다.

영향: 느린 배치가 5분을 넘으면 다른 worker가 아직 처리 중인 행을 재선점할 수 있습니다. 발송 후 종료·복구가 반복되면 실패 처리 경로의 5회 제한만으로 재전송을 제한하지 못합니다.

제안: 1회 선점 수·최악 발송 시간을 함께 조정하고, lease 갱신 또는 소량 선점·선점 세대 토큰을 검토합니다. stale 복구 횟수/최대 이벤트 수명을 별도 제한합니다. 소유권 검사는 DB 쓰기를 제한할 뿐 이미 발생한 외부 발송을 취소하지 못합니다.

우선 1: 운영 보안 경계

관찰: 테스트 비활성 보안 체인은 anyRequest().permitAll()이고 활성 체인도 /internal/** 외에는 permitAll입니다. 이력·기기 Controller는 Principal을 자체 검사하지만 Actuator 접근 제한은 이 코드에서 제공하지 않습니다.

제안: 회사 인증과 관리 엔드포인트 접근 정책을 명시합니다. /api/**의 CSRF 제외는 채택할 인증 방식과 함께 검토합니다. 외부 접근 가능 여부는 실제 리버스 프록시·방화벽 배치도 확인해야 합니다.

우선 2: 플랫폼·시도 횟수 이력 정확성

관찰: PushProcessor가 완료 저장에 Platform.IOS를 고정 전달하며, findByEventId()는 완료 이력의 attemptCount를 1로 반환합니다. Topic은 양 플랫폼으로 전달될 수 있습니다.

제안: 플랫폼을 단말 통계로 해석하지 않도록 TOPIC/MULTI 또는 별도 채널 모델로 이관하고 DB check 제약도 변경합니다. 완료 시 실제 시도 횟수와 큐 생성 시각을 저장해 화면에 표시합니다.

9. 기능 변경 설계와 권장 작업 순서

다음으로 결정할 기능

1. 전송 문구 감사: 현재 history에 title/body/templateVersion이 없습니다. 재시도마다 최신 템플릿을 쓸지 최초 문구를 고정할지 결정하고 스냅샷 저장 정책을 만듭니다. displayName()과 실제 발송 템플릿도 별도로 존재하므로 관리 UI 문구와 함께 검토합니다.
2. 맥등성과 DEAD 재처리: queue/history의 unique는 테이블별입니다. 완료 이벤트 재등록 방지 정책, 관리자의 재처리 권한·사유·감사 로그, 기존 eventId 유지 여부를 설계합니다. 현재 전용 DEAD 재처리 API와 보관 정리 작업은 없습니다.
3. 퇴사·부서 변경·기기 분실: QR 서명은 정식 앱의 위조 검증이지 Firebase의 사용자별 구독 권한 통제가 아닙니다. nonce의 일회 사용 확인도 없습니다. 기존 구독자를 제외하려면 Topic 회전·재등록 등 별도 운영 설계가 필요합니다. 비활성 binding을 다시 QR 발급하면 기존 unique 행과 충돌해 할당 실패할 수 있어 재활성화 정책도 정해야 합니다.
4. 회사 통합: BusinessDirectory 구현, 관리자와 임직원 QR 발급 권한 분리, 회사 Firebase 전환, 인스턴스 공통 요청 제한, 이력 보관 기간을 결정합니다.

트랜잭션 변경 시 주의

enqueueUser/Department/Notice는 내부에서 같은 객체의 @Transactional 메서드를 호출합니다. Spring 프록시 self-invocation에는 트랜잭션이 새로 적용되지 않습니다. 현재 단일 INSERT는 실행되지만 업무 원자성은 호출자 트랜잭션이 보장해야 합니다. 공개 진입점에 일관된 경계를 두고, 다른 Bean을 통한 호출·업무 롤백 통합 테스트로 확인하세요.

권장 변경 묶음

순서	작업	완료 조건
1	이력 대상 분리·플랫폼 정정	ID 충돌·양 플랫폼 이력 테스트
2	stale·소유권·맥등성 정책	느린 worker·중단 복구·완료 재등록 테스트
3	운영 인증·QR 생명주기	권한 거부·퇴사/부서 변경 시나리오
4	문구·보관·재처리 기능	스냅샷·보관 정리·감사 이력 검증
5	화면·새 타입·처리량 확장	앱 호환성·부하·실기 검증

변경 요청은 “현재 동작 / 원하는 동작 / 대상 사용자 / 실패 시 처리 / 기존 데이터 이관 / 앱 재배포 여부 / 완료 기준”으로 작성하면 구현 범위를 결정하기 쉽습니다.

검토 예: “문구 변경 이후에도 기존 발송 문구를 조회하고 싶다”면 새 history 컬럼·스냅샷 저장 시점·과거 데이터 표시 정책까지 함께 정합니다. 소스 확인 기반 후보이며 수정 승인은 별도 작업 요청에서 정하면 됩니다.

참조 문서: docs/API.md, docs/ARCHITECTURE.md, docs/SECURITY.md, docs/COMPANY-DEPLOYMENT.md 및 push-gateway/README.md. 코드 근거는 각 장에 명시한 클래스·메서드를 기준으로 확인하세요.